

AI-Augmented BlogNest Backend using Node.js

Project documentation-Team 1

1. Introduction :

- **Project Title:** AI BlogNest API – AI-Augmented Blog Application
- **Team ID:** SWTID-2026-1876

- **Team Leader**
- **Name:** Ayesha Thahasin F
- **Email ID:** 24bsccs30@cttewc.edu.in

- **Team Member 1:**
- **Name:** Karthiga S
- **Email ID:** 24bsccs40@cttewc.edu.in

- **Team Member 2:**
- **Name:** Prithika SV
- **Email ID:** 24bsccs50@cttewc.edu.in

- **Team Member 3:**
- **Name:** Dheva Dharshini. D
- **Email ID:** 24bsccs12@cttewc.edu.in

Introduction:

AI BlogNest API is an AI-augmented backend application developed for managing blog content and providing AI-based blog services. The application provides user authentication, blog management, blog fetching, AI-based blog generation, and blog summarization.

The backend is developed using Node.js, Express.js, MongoDB, Mongoose, and Google Gemini AI. The application follows a structured architecture using controllers, middleware, models, routes, and services.

2. Project Overview:

AI BlogNest API is a RESTful backend application designed to provide a platform for creating and managing blog content with the support of Artificial Intelligence.

The application allows users to register and log in securely, manage blog posts, retrieve blog information, generate new blog content using Gemini AI, and summarize existing blog content.

The project is organized into different modules such as authentication, blog management, AI services, database models, routes, and middleware. This modular structure makes the application easier to develop, maintain, and extend.

3. Objectives of the Project:

- To develop a structured backend API for a blog application.
- To provide secure user registration and login functionality.
- To implement authentication using JWT.
- To securely store user passwords using bcrypt.
- To provide blog creation and management functionality.
- To retrieve and manage blog data using MongoDB.
- To integrate Google Gemini AI with the backend application.
- To generate blog content automatically using AI.
- To summarize existing blog content using AI.
- To develop a modular and maintainable backend system.
- To provide APIs that can be easily tested using API testing tools.

4. Technologies Used:

Node.js – Node.js is used as the runtime environment for executing the backend application.

Express.js – Express.js is used to create the server and develop RESTful APIs.

MongoDB – MongoDB is used as the database for storing user and blog information.

Mongoose – Mongoose is used to connect the Node.js application with MongoDB and manage database models.

Google Gemini AI – Google Gemini AI is integrated to provide AI-powered blog generation and blog summarization.

JWT – JSON Web Token is used for user authentication and protecting secured API operations.

bcrypt – bcrypt is used for securely hashing user passwords.

dotenv – dotenv is used to manage environment variables such as the MongoDB connection, JWT secret, and Gemini API configuration.

Thunder Client – Thunder Client is used to test and verify the REST APIs.

Visual Studio Code – Visual Studio Code is used as the development environment.

GitHub – GitHub is used for storing and managing the project source code.

5. System Architecture:

AI BlogNest API follows a modular backend architecture. The client sends requests to the Express.js server through REST APIs. The request is processed through the required middleware and routes and then handled by the appropriate application logic.

The controllers handle the required operations, while models communicate with the MongoDB database through Mongoose. AI-related operations are handled through the services layer, which connects the application with Gemini AI.

System Flow

Client → Express.js Server → Routes → Middleware → Controllers/Services → MongoDB or Gemini AI → Response

This architecture separates different responsibilities of the application and makes the backend easier to maintain and extend.

6. Folder Structure:

AI BlogNest API

```
|
├── src
|   ├── controllers
|   |   └── blogController.js
|   |
|   ├── middleware
|   |   ├── authMiddleware.js
|   |   └── errorMiddleware.js
|   |
|   ├── models
|   |   ├── Blog.js
|   |   └── User.js
|   |
|   ├── routes
|   |   ├── aiRoutes.js
|   |   ├── authRoutes.js
|   |   └── blogRoutes.js
|   |
|   └── services
|
├── app.js
└── server.js

├── .env
├── .gitignore
└── package.json
```

```
|— package-lock.json
|— README.md
|— Thunder Client API Collection
└— Thunder Client Environment
```

7. Features:

- User Authentication
- User registration
- User login
- Password protection using bcrypt
- JWT-based authentication
- Protected API access
- Blog Management
- Create blog posts
- View blog posts
- Update blog posts
- Delete blog posts
- Fetch blog information
- AI Features
- AI-powered blog generation
- AI-powered blog summarization
- Integration with Google Gemini AI
- Database Management
- MongoDB database integration
- Mongoose models
- User data management
- Blog data management
- Error Handling
- Centralized error-handling middleware
- Proper API response handling
- API Testing
- REST API testing using Thunder Client
- JSON-based request and response handling

8. Project Setup:

Step 1: Install Node.js – Node.js is installed on the system to run the backend application.

Step 2: Open the Project – The AI BlogNest API project is opened in Visual Studio Code.

Step 3: Install Dependencies – The required project dependencies are installed using the npm package manager.

Step 4: Configure Environment Variables – The required environment variables are configured in the .env file. These include the server port, MongoDB connection string, JWT secret, and Gemini AI configuration.

Step 5: Configure MongoDB – The MongoDB connection string is configured so that the backend application can connect to the database.

Step 6: Start the Application – The backend server is started using the configured npm development command.

Step 7: Test the APIs – After the server starts successfully, the APIs are tested using Thunder Client.

9. API Development:

The AI BlogNest API is developed using Express.js and follows a RESTful API structure. The API functionality is separated into authentication, blog management, and AI-related operations.

9.1 Register API

The Register API is used to create a new user account. It receives the required user information, validates the details, securely processes the password, and stores the user information in the MongoDB database.

9.2 Login API

The Login API is used to authenticate registered users. It verifies the user's login credentials and generates a JWT token after successful authentication. The token is used to access protected API operations.

9.3 Blogs API

The Blogs API is responsible for managing blog posts. It provides functionality for creating, viewing, updating, and deleting blog posts. Authentication is applied where required to protect blog operations.

9.4 Generate Blog API

The Generate Blog API provides an AI-powered blog creation feature. The required topic and category information are processed and sent to the Gemini AI service. The generated content is then returned through the API response.

9.5 Summarize Blog API

The Summarize Blog API uses Gemini AI to generate a short and meaningful summary from existing blog content. This helps reduce lengthy content into its main points.

9.6 Fetch Blog API

The Fetch Blog API retrieves stored blog information from the MongoDB database. It allows the application to access and display the required blog content through the API.

The API routes are organized separately using `aiRoutes.js`, `authRoutes.js`, and `blogRoutes.js`. This separation keeps the API structure organized and makes the application easier to maintain.

10. Middlewares:

Middleware functions are used to process requests before they reach the main application logic.

Authentication Middleware – The `authMiddleware.js` file is used to handle authentication for protected routes. It verifies the JWT token provided by the user and allows authorized requests to continue.

Error Middleware – The `errorMiddleware.js` file is used to handle errors occurring during API execution. It provides a centralized approach for handling application errors and returning appropriate responses.

Middleware Flow

Client Request → Authentication Middleware → Route → Controller/Service → Response

Middleware improves the security, reliability, and organization of the backend application.

11. Database Development:

MongoDB is used as the database for the AI BlogNest API. Mongoose is used to establish communication between the Node.js application and MongoDB.

The project contains two main database models:

User Model – The `User.js` model is used to define and manage user-related information required for authentication and user management.

Blog Model – The Blog.js model is used to define and manage blog-related information such as blog content and other required details.

12. Project Execution:

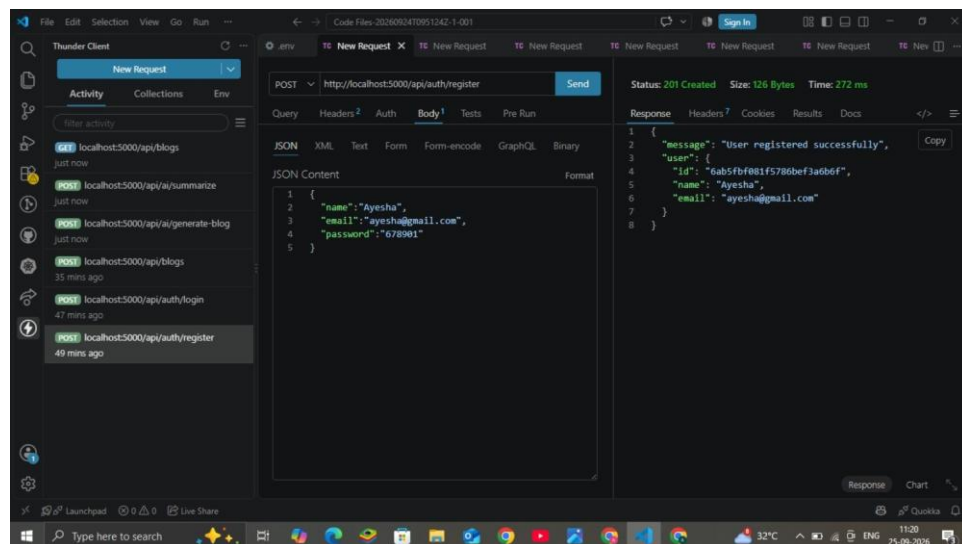
The project execution starts by running the backend server from Visual Studio Code.

After starting the server:

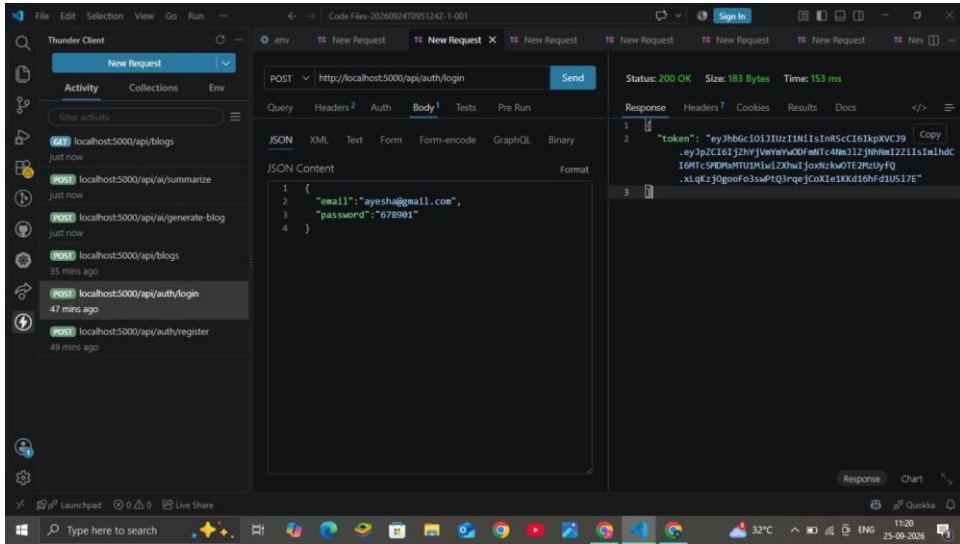
- The Express.js application is initialized.
- The server starts on the configured port.
- The application establishes a connection with MongoDB.
- API routes become available.
- Authentication APIs can be tested.
- Blog APIs can be tested.
- AI APIs can be tested using Gemini AI.
- API responses are verified using Thunder Client.

13. Output Screenshots:

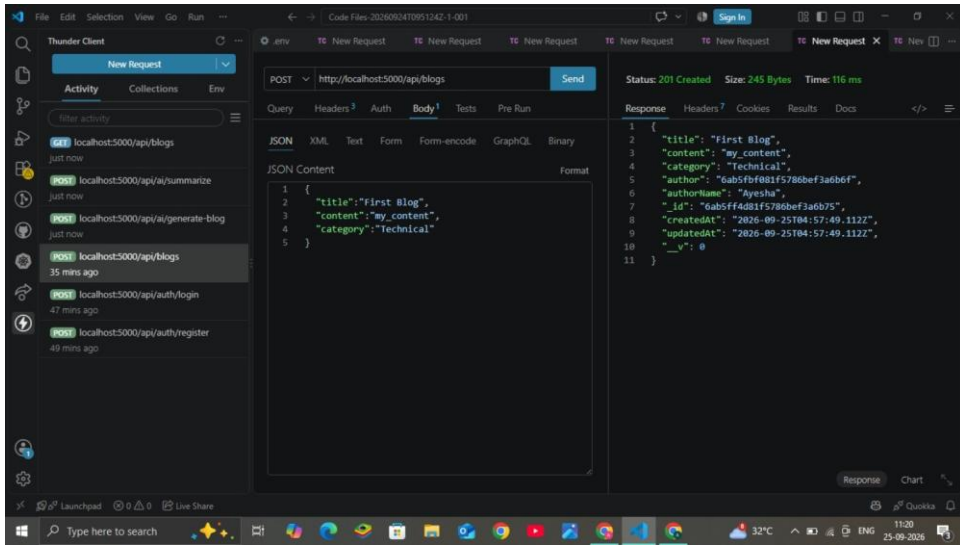
Register output



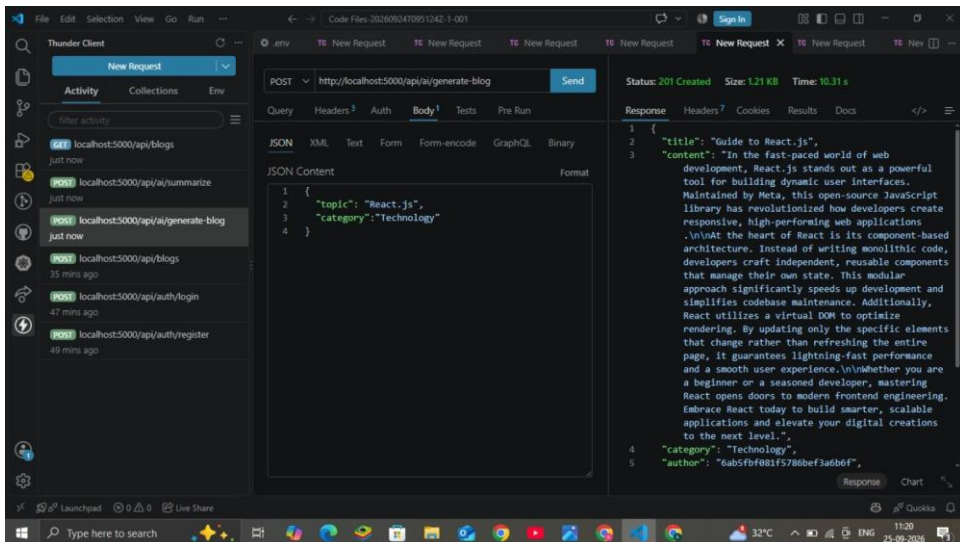
Login output



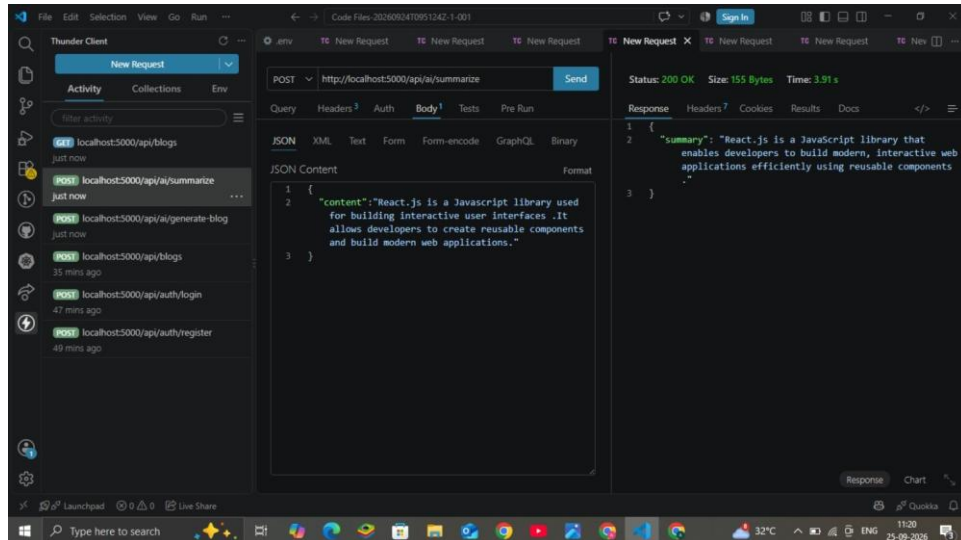
Create Blog output



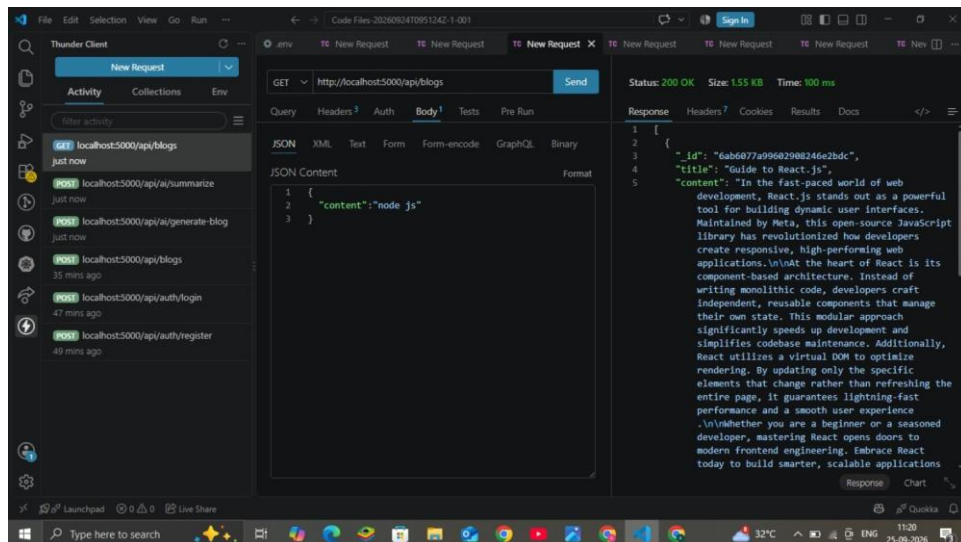
Generate Blog output



Summarize Blog output



Get Blog output



14. Demo Video link:

https://drive.google.com/file/d/1MhCqIg10Q5YqpqMNdZpT8p_pwEBKDYQM/view?usp=drive_sdk

The demo video shows the working of the AI BlogNest API, including:

- User registration
- User login
- Blog management
- AI-powered blog generation
- AI-powered blog summarization

15. Future: Enhancement:

- Development of a user-friendly frontend application.
- AI-based blog title generation.
- AI-based grammar and content improvement.
- AI-generated images for blog posts.
- Advanced blog search and filtering.
- Categories and tags for better blog organization.
- Comments and likes for user interaction.
- User profile management.
- Admin dashboard for managing users and blogs.
- Personalized AI-based blog recommendations.
- Cloud deployment of the backend application.
- Additional AI-powered content management features.